

Universidad de la República

Facultad de Ingeniería

Maestría en Ciencia de Datos y Aprendizaje Automático

Tarea 2

Introducción a la Ciencia de Datos

Natalia Fitipaldi
Valentina Torres

25 de June de 2026

Introducción

En este trabajo se utilizará el mismo conjunto de datos aplicado en la Tarea 1, derivado del dataset abierto All the News 2.1 - Component One, que reúne artículos de noticias publicados por distintos medios de prensa estadounidenses entre 2016 y 2020. A partir de este conjunto, se trabajará con un subconjunto compuesto por los tres medios con mayor cantidad de artículos, utilizando el texto de las noticias como principal fuente de información.

El objetivo de esta tarea es aplicar técnicas de aprendizaje automático sobre datos de texto. Para ello, se transformarán los artículos a formatos numéricos que puedan ser utilizados por distintos modelos de clasificación, con el fin de predecir a qué medio de prensa pertenece cada noticia a partir de su contenido.

Además, se analizarán diferentes formas de representar el texto, se utilizarán técnicas de reducción de dimensionalidad para visualizar los datos y se evaluará el desempeño de distintos modelos de clasificación. Finalmente, se discutirán los resultados obtenidos, las limitaciones de los métodos utilizados y posibles factores que pueden influir en la capacidad de distinguir entre medios de prensa a partir de sus artículos.

Parte 1: Dataset y representación numérica de texto

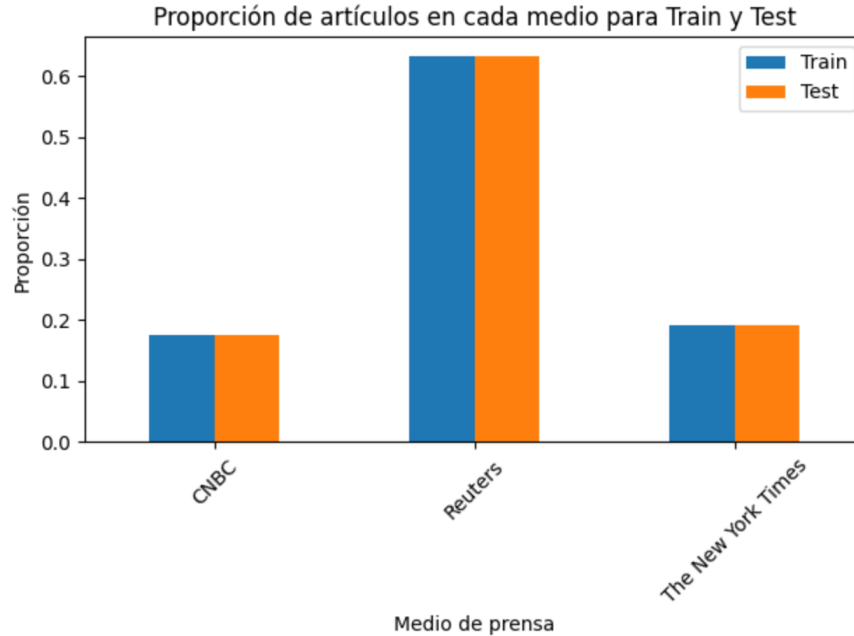
1. Preparación del dataset

En primer lugar, se seleccionaron los tres medios de prensa con mayor cantidad de artículos dentro del conjunto de datos. Luego, se aplicó la función `clean_text()` desarrollada en la Tarea 1 para realizar la limpieza y normalización de los textos.

Posteriormente, el conjunto de datos se dividió en un conjunto de entrenamiento (70%) y un conjunto de prueba (30%) mediante la función `train_test_split` de `scikit-learn`. Se utilizó muestreo estratificado (`stratify`) tomando como referencia el medio de prensa, con el objetivo de mantener aproximadamente la misma proporción de artículos de cada categoría en ambos conjuntos. Además, se fijó un valor para `random_state` para garantizar la reproducibilidad de los resultados.

2. Verificación del balance de clases

Publication	Total (n)	Train (n)	Test (n)	Train (%)	Test (%)
Reuters	9431	6601	2830	63.3%	63.3%
The New York Times	2840	1988	852	19.1%	19.1%
CNBC	2623	1836	787	17.6%	17.6%
TOTAL	14894	10425	4469	100.0%	100.0%



El gráfico y la tabla permiten verificar que la distribución de artículos por medio se conserva en los conjuntos de entrenamiento y prueba. *Reuters* es el medio con mayor cantidad de artículos (63,3%), seguido por *The New York Times* (19,1%) y *CNBC* (17,6%). Se observa que estas proporciones se mantienen prácticamente idénticas en ambas particiones, lo que confirma que el muestreo estratificado preservó el balance original de las clases. Esto es importante porque asegura que el conjunto de prueba sea representativo de los datos utilizados para entrenar los modelos, permitiendo una evaluación más confiable de su desempeño.

3. Representación Bag of Words

Los algoritmos de aprendizaje automático no pueden trabajar directamente con texto, por lo que es necesario transformarlo a una representación numérica. Una de las técnicas más simples para hacerlo es Bag of Words (BoW).

Esta técnica consiste en construir un vocabulario con todas las palabras que aparecen en el conjunto de entrenamiento y representar cada documento mediante un vector que indica cuántas veces aparece cada palabra. De esta forma, cada columna de la matriz corresponde a una palabra del vocabulario y cada fila corresponde a un documento.

Por ejemplo, si el vocabulario estuviera formado por las palabras (“president”, “senate”, “government”), el texto “president senate senate” se representaría como (1, 2, 0), mientras que el texto “president government” se representaría como (1, 0, 1). El resultado es una matriz numérica que resume la frecuencia de aparición de cada término en cada documento.

Una característica importante de Bag of Words es que no tiene en cuenta el orden de las palabras dentro del texto, sino únicamente su presencia o frecuencia. Esto simplifica considerablemente la representación, aunque también implica la pérdida de parte de la información contextual contenida en los documentos.

Al aplicar esta transformación sobre el conjunto de entrenamiento se obtuvo una matriz de dimensiones (10425, 82827). Esto significa que la matriz contiene 10.425 documentos (filas) y un vocabulario de 82.827 palabras distintas (columnas). Cada celda indica la cantidad de veces que una determinada palabra aparece en un determinado documento.

La matriz resultante es una matriz sparse (dispersa) de tipo `csr_matrix`. En total contiene 863.471.475 celdas, pero solamente 2.136.292 valores distintos de cero, lo que representa apenas 0,2474% del total. En otras palabras, más del 99,7% de los elementos de la matriz son ceros.

Esto ocurre porque cada artículo utiliza únicamente una pequeña parte del vocabulario disponible. Aunque el vocabulario contiene más de 82 mil palabras, un documento individual solo incluye una fracción muy reducida de ellas. Debido a esta característica, almacenar la matriz completa de forma tradicional implicaría un uso innecesariamente elevado de memoria. Por este motivo, scikit-learn utiliza estructuras especiales para matrices dispersas que almacenan únicamente los valores distintos de cero y sus posiciones, reduciendo significativamente los requerimientos de memoria y permitiendo trabajar con conjuntos de datos mucho más grandes.

4. Representación TF-IDF

Un n-grama es una secuencia de n palabras consecutivas dentro de un texto. Cuando $n = 1$ se habla de unigramas, que corresponden a palabras individuales y cuando $n = 2$ se habla de bigramas, formados por pares de palabras consecutivas. La utilización de n-gramas permite incorporar parte del contexto y capturar expresiones frecuentes que podrían perderse al considerar únicamente palabras aisladas.

Luego de obtener la representación Bag of Words, se aplicó la transformación Term Frequency - Inverse Document Frequency (TF-IDF). Esta técnica asigna un peso a cada término teniendo en cuenta no solo cuántas veces aparece en un documento (Term Frequency), sino también cuán frecuente es en el conjunto total de documentos (Inverse Document Frequency).

TF (Term Frequency) se calcula como:

$$TF(t, d) = \frac{\text{cantidad de veces que aparece } t \text{ en } d}{\text{cantidad total de palabras en } d}$$

donde:

- **t** representa el término (palabra) considerado.
- **d** representa el documento.

IDF (Inverse Document Frequency) se calcula como:

$$IDF(t) = \log \left(\frac{N}{df(t)} \right)$$

donde:

- **N** es la cantidad total de documentos.
- **df(t)** es la cantidad de documentos que contienen el término **t**.

Finalmente, el peso TF-IDF se obtiene multiplicando ambos componentes:

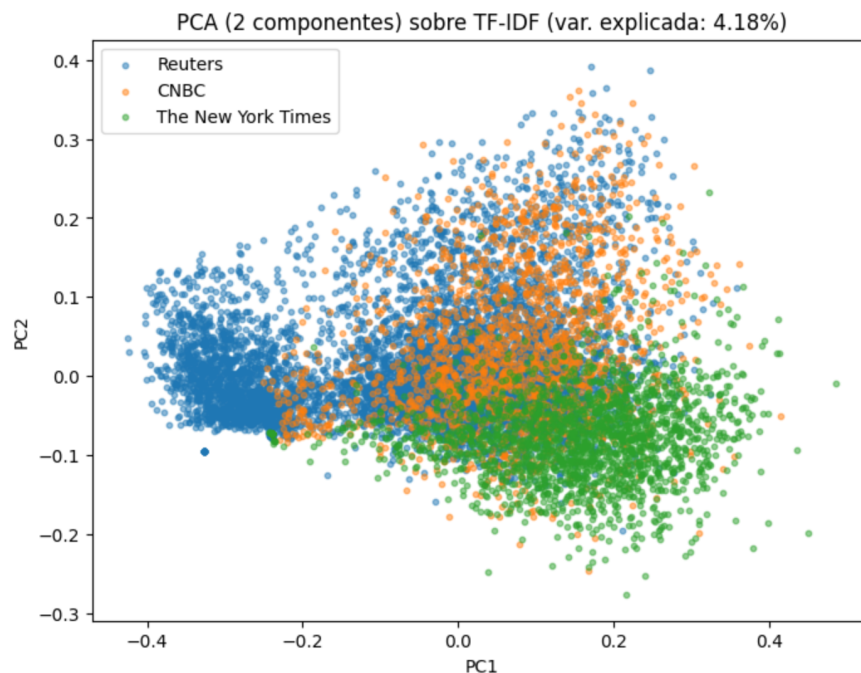
$$TF - IDF(t, d) = TF(t, d) \times IDF(t)$$

La idea principal es que las palabras muy frecuentes en todos los documentos suelen aportar poca información para distinguir entre ellos. Por ejemplo, términos como *the*, *and* o *of* aparecen en una gran cantidad de artículos y, por lo tanto, reciben un peso relativamente bajo. En cambio, palabras que aparecen con frecuencia en un documento particular pero no son comunes en el resto del corpus reciben un peso más alto.

A diferencia de Bag of Words, donde los valores representan conteos de palabras, en la representación TF-IDF los valores reflejan la importancia relativa de cada término dentro de cada documento. Esto suele generar una representación más útil para tareas de clasificación de texto.

5. Visualización PCA sobre TF-IDF

Para explorar la estructura de los datos, se aplicó Análisis de Componentes Principales (PCA) sobre los vectores TF-IDF del conjunto de entrenamiento y se representaron los documentos utilizando los dos primeros componentes principales. Dado que los vectores de texto tienen una dimensionalidad muy alta, PCA permite proyectarlos en un espacio bidimensional para facilitar su visualización, preservando la mayor cantidad posible de variabilidad de los datos.



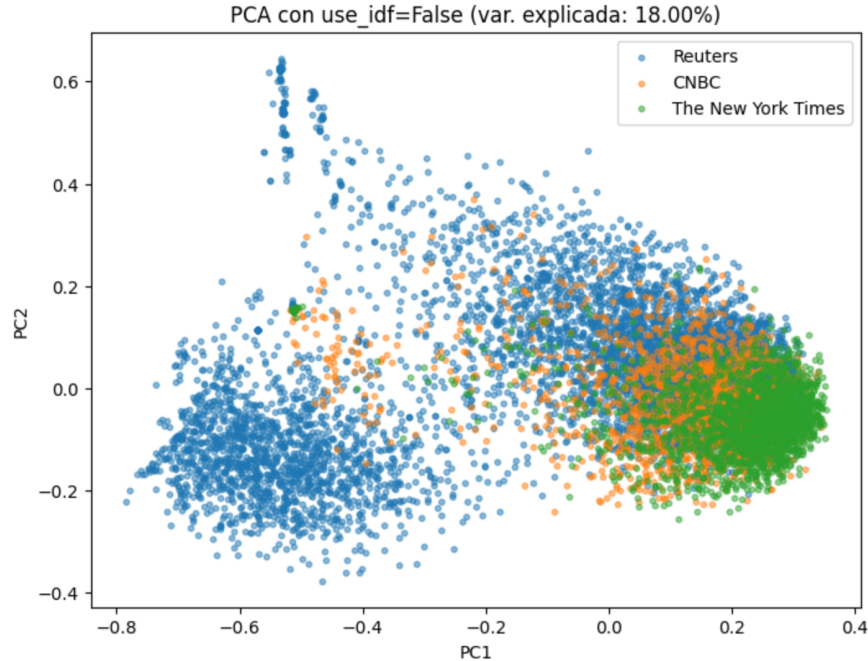
El gráfico muestra la proyección de los artículos utilizando los dos primeros componentes principales calculadas sobre los vectores TF-IDF. En este caso, los dos componentes explican aproximadamente el 4,18% de la varianza total, lo que indica que gran parte de la información del conjunto de datos permanece en dimensiones adicionales.

Aun así, se observan algunos patrones interesantes. Los artículos de *Reuters* tienden a concentrarse hacia los valores negativos del primer componente (PC1), formando un grupo relativamente diferenciado en el sector izquierdo de la figura. Por su parte, los artículos de *The New York Times* se ubican con mayor frecuencia en valores positivos de PC1 y negativos de PC2, mientras que *CNBC* ocupa principalmente la zona central y superior del gráfico.

Sin embargo, también existe un importante grado de superposición entre los tres medios, especialmente en la región central del mapa. Esto sugiere que, aunque pueden identificarse ciertas diferencias en el vocabulario utilizado por cada medio, las dos primeras componentes principales no son suficientes para separar claramente las categorías. En consecuencia, la clasificación de los artículos probablemente requiera considerar un número mayor de dimensiones que las representadas en esta visualización.

- `use_idf = FALSE`

El caso base utiliza TF-IDF con `use_idf=True`, que reduce el peso de las palabras que aparecen en muchos documentos. Para evaluar el efecto de esta ponderación, se realizó una segunda visualización utilizando `use_idf=False`, donde los términos se consideran únicamente según su frecuencia dentro de cada artículo.

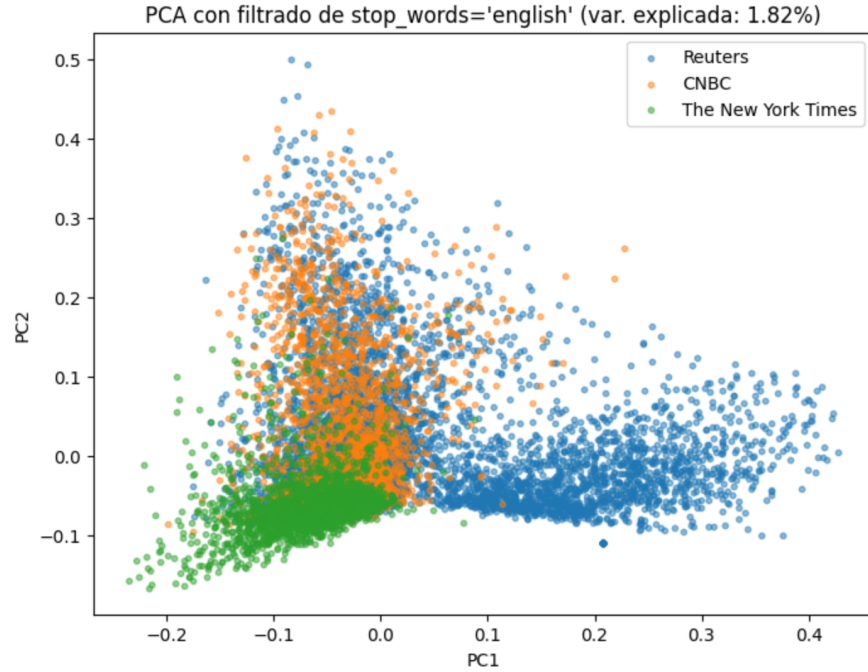


En el gráfico se observa la proyección obtenida al desactivar la componente IDF. En este caso, los dos primeros componentes explican aproximadamente el 18,0% de la varianza, un valor considerablemente mayor que el observado en el caso base (4,18%). Además, la separación visual entre algunos grupos parece más marcada, especialmente para los artículos de *Reuters*, que forman un conjunto relativamente diferenciado hacia los valores negativos de PC1.

Sin embargo, esta mayor separación no implica necesariamente una mejor representación para tareas de clasificación. Al utilizar `use_idf=False`, las palabras muy frecuentes adquieren más peso y pueden dominar la representación de los documentos. En cambio, TF-IDF busca destacar términos más específicos y potencialmente más útiles para distinguir entre medios. Por esta razón, aunque el mapa sin IDF muestra una estructura visual más definida y concentra una mayor proporción de la varianza en las primeras componentes, la representación TF-IDF suele ser más adecuada para los modelos de clasificación que se entrenarán posteriormente.

- Filtrado de stopwords para idioma inglés

A continuación, se evaluó el efecto de eliminar las stop words del idioma inglés mediante el parámetro `stop_words='english'`. Estas palabras corresponden a términos muy frecuentes, como artículos, preposiciones y conjunciones, que suelen aportar poca información para diferenciar documentos.

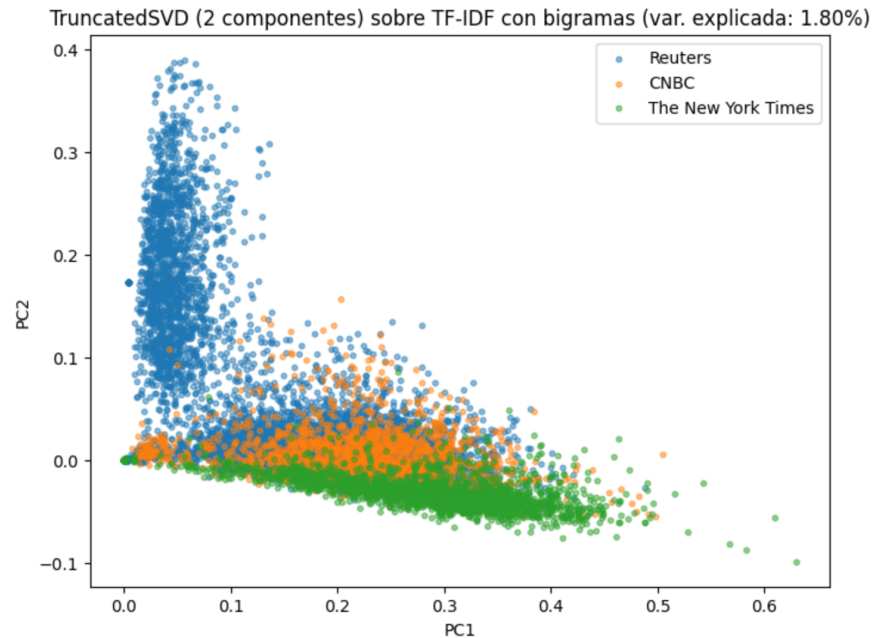


En el gráfico se observa la proyección obtenida luego de eliminar las stop words. En este caso, los dos primeros componentes explican aproximadamente el 1,82% de la varianza, un valor menor al observado en el caso base (4,18%).

Visualmente, se puede ver una separación más marcada de los artículos de *Reuters*, que tienden a concentrarse hacia valores positivos del primer componente (PC1). Sin embargo, los artículos de *CNBC* y *The New York Times* continúan presentando un importante grado de superposición, especialmente en la región central del gráfico.

La disminución de la varianza explicada sugiere que las palabras eliminadas contribuían significativamente a la estructura capturada por los primeros componentes principales. No obstante, muchas de estas palabras son muy comunes en todos los documentos y no necesariamente resultan útiles para distinguir entre medios. Por esta razón, aunque la representación bidimensional captura menos varianza, el filtrado de stop words puede seguir siendo beneficioso para tareas de clasificación al reducir ruido y enfocarse en términos más informativos.

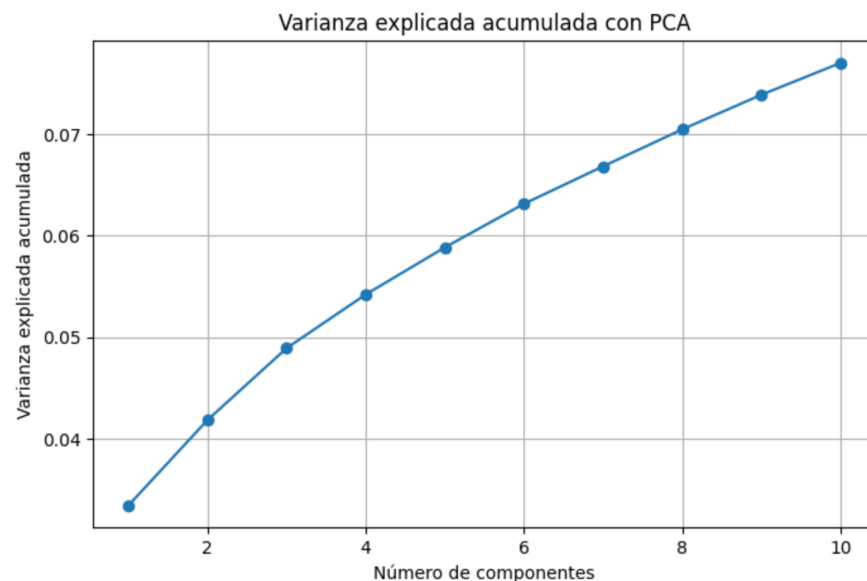
- `ngram_range = (1,2)`



El gráfico muestra la proyección obtenida al incorporar bigramas ($\text{ngram_range}=(1,2)$) en la representación TF-IDF. En comparación con el caso base, se observa una separación más marcada entre los artículos de *Reuters* y los de los otros dos medios. En particular, *Reuters* forma un grupo relativamente bien definido hacia valores bajos del primer componente (PC1) y altos del segundo componente (PC2).

Por otro lado, *CNBC* y *The New York Times* continúan presentando un importante grado de superposición, aunque se aprecia una ligera tendencia a que los artículos de *The New York Times* se concentren en valores más bajos del segundo componente (PC2).

Estos resultados sugieren que la incorporación de bigramas aporta información adicional sobre el contexto de las palabras, permitiendo capturar expresiones características que no pueden representarse utilizando únicamente palabras individuales. Esto parece favorecer la diferenciación de *Reuters* respecto a los demás medios, aunque los dos primeros componentes siguen sin ser suficientes para lograr una separación completa entre las tres categorías.



La varianza explicada acumulada crece de forma lenta a medida que se incorporan componentes. Incluso utilizando 10 componentes, solo se explica aproximadamente un 7–8% de la varianza total. Esto es consistente con la naturaleza de los datos textuales representados mediante TF-IDF, donde la información se encuentra distribuida en un espacio de alta dimensionalidad y gran dispersión.

A partir de los gráficos anteriores, no parece posible separar claramente los tres medios de prensa utilizando únicamente dos componentes. Si bien se observan ciertas zonas con predominio de algunos medios, especialmente en el caso de *Reuters*, también existe un solapamiento importante entre las categorías, sobre todo entre *CNBC* y *The New York Times*. Esto sugiere que la información útil para distinguir entre medios no se concentra en solo dos dimensiones, sino que está distribuida entre muchas características del texto.

La separación parcial observada podría deberse a distintos factores. Por un lado, pueden existir diferencias de estilo editorial, como la forma de redactar los titulares, el uso de determinadas expresiones o el tono general de los artículos. Por otro lado, también pueden influir diferencias temáticas entre los medios.

Además, no se puede descartar que parte de la separación observada se deba a pistas explícitas del medio que no hayan sido removidas durante la limpieza del texto. En esta tarea se reutilizó la función `clean_text()` desarrollada en la Tarea 1, que convierte el texto a minúsculas, elimina signos de puntuación, números y el contenido inicial hasta el primer salto de línea. Sin embargo, esta limpieza no elimina de forma específica nombres de medios, nombres de autores, secciones u otras expresiones recurrentes que podrían aparecer dentro del cuerpo de los artículos.

Por lo tanto, la separación observada no necesariamente refleja únicamente diferencias profundas de estilo editorial, sino también posibles diferencias temáticas o patrones propios del contenido original que permanecieron luego del preprocesamiento aplicado.

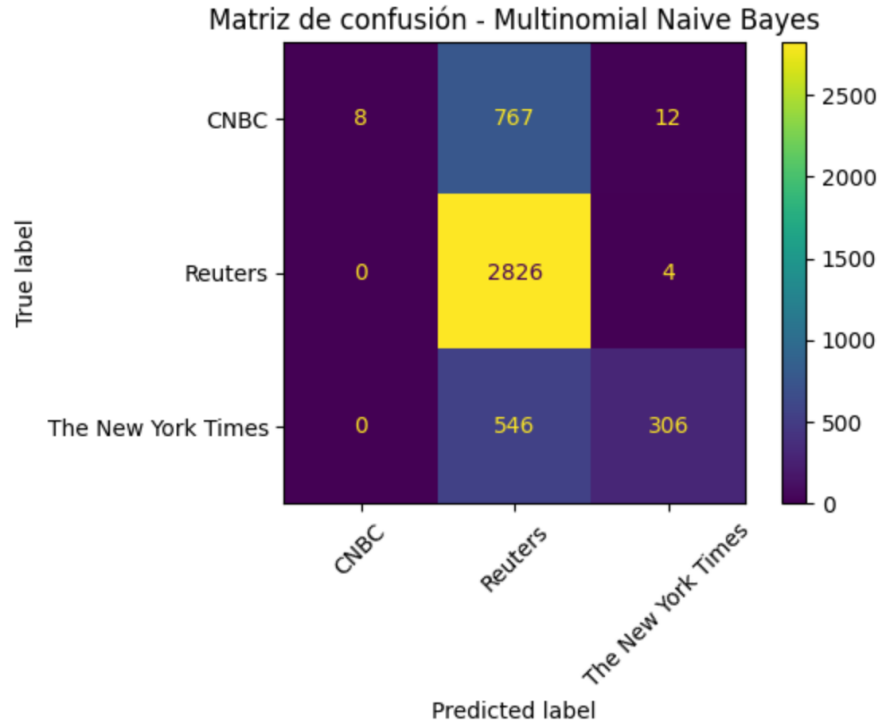
Parte 2: Entrenamiento y Evaluación de Modelos

1. Multinomial Naive Bayes

Multinomial Naive Bayes es un algoritmo de clasificación probabilístico basado en el teorema de Bayes. En su versión multinomial, asume que cada documento puede representarse mediante la frecuencia de aparición de sus palabras (o sus pesos TF-IDF) y estima la probabilidad de que pertenezca a cada clase utilizando esa información.

Como primer modelo de clasificación, se entrenó un modelo Multinomial Naive Bayes utilizando el conjunto de entrenamiento representado mediante TF-IDF. Posteriormente, el modelo se aplicó sobre el conjunto de prueba para predecir el medio de prensa de cada artículo y evaluar su desempeño mediante el accuracy, la matriz de confusión y las métricas de precision y recall para cada clase.

	Precision	Recall	F1-score	Support
CNBC	1.00	0.01	0.02	787
Reuters	0.68	1.00	0.81	2830
The New York Times	0.95	0.36	0.52	852
Accuracy			0.70	4469
Macro avg	0.88	0.46	0.45	4469
Weighted avg	0.79	0.70	0.62	4469



La matriz de confusión muestra que el modelo clasifica correctamente la gran mayoría de los artículos de *Reuters*, con 2826 predicciones correctas sobre un total de 2830. En cambio, el desempeño es considerablemente peor para *CNBC* y *The New York Times*. En particular, la mayoría de los artículos de *CNBC* (767 de 787) fueron clasificados incorrectamente como *Reuters*, mientras que en *The New York Times* también se observa una cantidad importante de errores (546 de 852), principalmente hacia la misma clase.

Estos resultados indican que el modelo presenta una fuerte tendencia a predecir *Reuters*, lo que probablemente esté influenciado por el desbalance existente en el conjunto de datos, donde este medio representa la mayor proporción de artículos.

Las métricas de precisión y recall reflejan el comportamiento observado en la matriz de confusión. *Reuters* presenta un recall cercano a 1 (1,00), ya que prácticamente todos sus artículos fueron clasificados correctamente. Sin embargo, su precisión es menor (0,68) porque muchos artículos pertenecientes a *CNBC* y *The New York Times* fueron clasificados erróneamente como *Reuters*.

En el caso de *CNBC* ocurre la situación opuesta. Su precisión es muy alta (1,00), lo que indica que cuando el modelo predice *CNBC* prácticamente siempre acierta. Sin embargo, su recall es extremadamente bajo (0,01), ya que el modelo identifica correctamente solo una pequeña fracción de los artículos de este medio y clasifica la mayoría como *Reuters*.

Para *The New York Times*, tanto la precisión (0,95) como el recall (0,36) muestran un comportamiento intermedio. Aunque las predicciones realizadas como *The New York Times* suelen ser correctas, el modelo deja sin identificar una parte importante de sus artículos, clasificándolos principalmente como *Reuters*.

El accuracy por sí solo no es suficiente para evaluar el desempeño del modelo, ya que no permite identificar diferencias en el comportamiento entre clases. En este caso, el conjunto de datos presenta un desbalance entre medios (con *Reuters* representando aproximadamente el 63% de los artículos), por lo que un modelo puede obtener un accuracy relativamente alto favoreciendo la predicción de la clase mayoritaria. Sin embargo, esto no garantiza un buen desempeño sobre los medios minoritarios. En efecto, aunque el modelo alcanza un accuracy de 70,3%, presenta un recall de apenas 1% para *CNBC* y de 36% para *The New York Times*. Por esta razón, resulta necesario complementar la evaluación con la matriz de confusión y métricas como

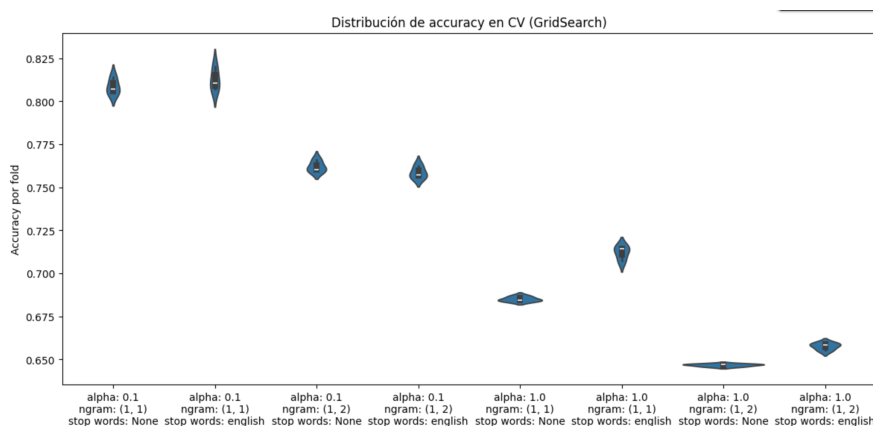
precision y recall, que permiten analizar el rendimiento del modelo para cada clase y detectar posibles sesgos hacia los medios más frecuentes.

Un aspecto que también podría contribuir a estos resultados es que *CNBC* suele publicar artículos provenientes de agencias de noticias como *Reuters*. Si algunos de estos artículos están presentes en el conjunto de datos, es esperable que compartan gran parte del vocabulario y del estilo de redacción, dificultando que el modelo los diferencie. Esto podría explicar, al menos en parte, la gran cantidad de artículos de *CNBC* clasificados como *Reuters*.

2. Validación cruzada y búsqueda de hiperparámetros

La validación cruzada (cross-validation) es una técnica para estimar el rendimiento real de un modelo de machine learning que consiste en dividir el conjunto de entrenamiento en varios subconjuntos llamados folds. En el esquema más común, k-fold cross-validation, el dataset se separa en k partes (por ejemplo, 5), y el modelo se entrena k veces, utilizando en cada iteración un fold distinto como conjunto de validación y los restantes como conjunto de entrenamiento. De esta forma se obtienen k valores de la métrica de evaluación (como accuracy), que luego se promedian para obtener una estimación más estable del desempeño del modelo. La principal ventaja de este enfoque es que no depende de una única partición train/test, sino que evalúa el modelo en múltiples divisiones de los datos, reduciendo la varianza de la estimación y proporcionando una medida más robusta de su rendimiento.

Para seleccionar los mejores hiperparámetros del modelo se utilizó GridSearchCV, una herramienta de scikit-learn que combina la búsqueda exhaustiva sobre una grilla de hiperparámetros con validación cruzada. De esta forma, cada combinación de parámetros es evaluada utilizando los distintos folds y se calcula el desempeño promedio obtenido. Finalmente, se selecciona automáticamente la configuración que alcanza la mejor métrica promedio.



El gráfico de validación cruzada muestra el desempeño de las distintas combinaciones de hiperparámetros evaluadas mediante GridSearchCV. La mejor configuración corresponde a $\alpha = 0.1$, unigramas ($\text{ngram_range}=(1,1)$) y eliminación de stop words, alcanzando un accuracy promedio de aproximadamente 0,813.

En general, las configuraciones con $\alpha = 0.1$ obtuvieron mejores resultados que aquellas con $\alpha = 1.0$, lo que sugiere que un menor nivel de suavizado se adapta mejor a este conjunto de datos. Asimismo, la eliminación de stop words produjo una mejora consistente en el desempeño, mientras que la incorporación de bigramas ($\text{ngram_range}=(1,2)$) no aportó mejoras y, en la mayoría de los casos, redujo el accuracy obtenido.

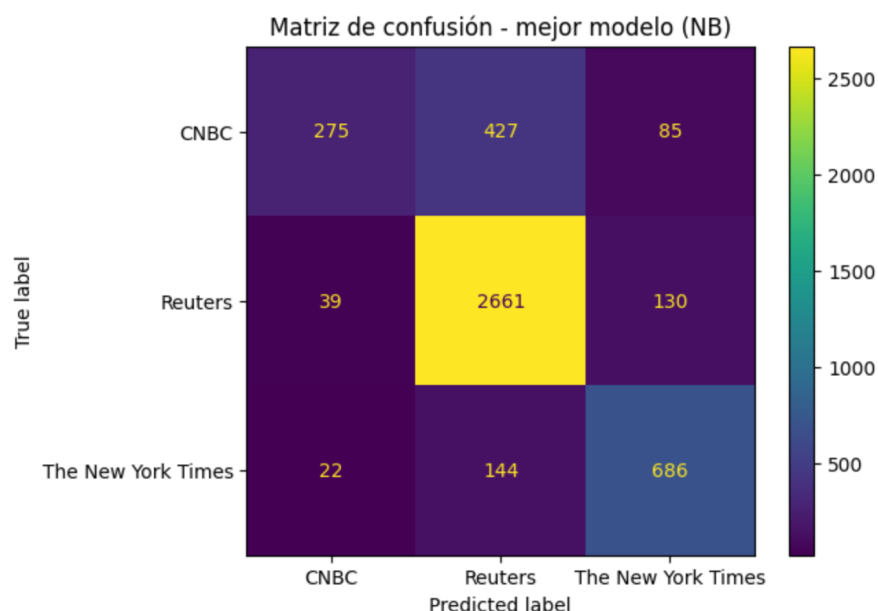
Finalmente, se observa una baja variabilidad entre los distintos folds para todas las configuraciones evaluadas. Esto indica que el desempeño del modelo es relativamente estable frente a diferentes particiones del conjunto de entrenamiento y que la estimación obtenida mediante validación cruzada resulta consistente.

3. Entrenamiento final con el mejor modelo

Una vez seleccionada la mejor combinación de hiperparámetros mediante GridSearchCV, el modelo se volvió a entrenar utilizando todo el conjunto de entrenamiento disponible. De esta forma, el algoritmo aprovecha la totalidad de los datos para ajustar sus parámetros antes de realizar la evaluación final sobre el conjunto de prueba.

	Precision	Recall	F1-score	Support
CNBC	0.82	0.35	0.49	787
Reuters	0.82	0.94	0.88	2830
The New York Times	0.76	0.81	0.78	852
Accuracy			0.81	4469
Macro avg	0.80	0.70	0.72	4469
Weighted avg	0.81	0.81	0.79	4469

Table 1: Accuracy: 0.8104



El modelo obtuvo un accuracy de aproximadamente 0,81, mejorando el desempeño respecto al modelo inicial. La matriz de confusión muestra que *Reuters* continúa siendo la clase mejor identificada, con un recall de 0,94. Sin embargo, también se observa una mejora importante en las clases *CNBC* y *The New York Times*, especialmente en sus valores de recall, indicando que el modelo logra reconocer una mayor proporción de artículos pertenecientes a estos medios.

A pesar de esta mejora, todavía existe una tendencia a clasificar algunos artículos de *CNBC* y *The New York Times* como *Reuters*. Esto puede estar asociado tanto al desbalance presente en el conjunto de datos como a similitudes en el vocabulario y el contenido de los artículos publicados por estos medios.

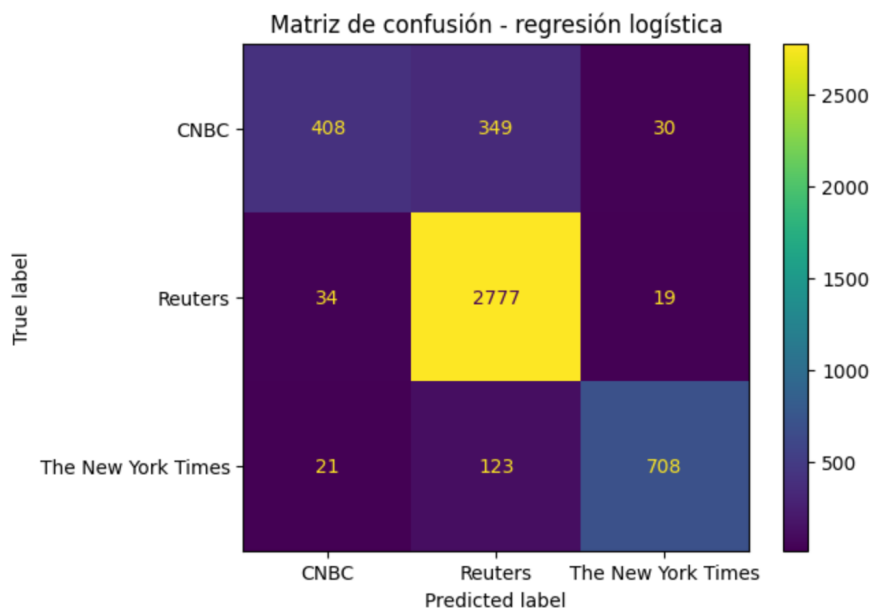
4. Modelo alternativo

Como modelo alternativo se evaluó la regresión logística, un algoritmo de clasificación lineal que estima, para cada documento, la probabilidad de pertenecer a cada una de las clases a partir de las características de entrada. A diferencia de Multinomial Naive Bayes, este modelo no asume que las palabras son independientes

	Precision	Recall	F1-score	Support
CNBC	0.88	0.52	0.65	787
Reuters	0.85	0.98	0.91	2830
The New York Times	0.94	0.83	0.88	852
Accuracy			0.87	4469
Macro avg	0.89	0.78	0.82	4469
Weighted avg	0.87	0.87	0.86	4469

Table 2: Accuracy: 0.8711

entre sí, sino que aprende durante el entrenamiento los pesos que mejor permiten diferenciar las distintas clases.



El modelo de Logistic Regression obtuvo un accuracy de aproximadamente 0,87, superando al mejor modelo basado en Multinomial Naive Bayes, que alcanzó un valor cercano a 0,81.

La matriz de confusión muestra que *Reuters* continúa siendo la clase con mejor desempeño, con un recall de 0,98. Sin embargo, también se observa una mejora importante en la clasificación de *CNBC* y *The New York Times*. En particular, *CNBC* aumenta su recall de 0,35 a 0,52, mientras que *The New York Times* alcanza un recall de 0,83. Aunque todavía existe una tendencia a clasificar algunos artículos de *CNBC* como *Reuters*, la cantidad de errores disminuye respecto al modelo anterior.

Métrica	Multinomial Naive Bayes	Regresión logística
Accuracy	0.81	0.87
Precision (CNBC)	0.82	0.88
Recall (CNBC)	0.35	0.52
Precision (Reuters)	0.82	0.85
Recall (Reuters)	0.94	0.98
Precision (The New York Times)	0.76	0.94
Recall (The New York Times)	0.81	0.83

En general, estos resultados muestran que la regresión logística obtiene un mejor desempeño que Multinomial Naive Bayes utilizando las mismas características de entrada. La mejora no solo se observa en el accuracy, sino también en una clasificación más equilibrada de los tres medios de prensa, especialmente en el caso de CNBC.

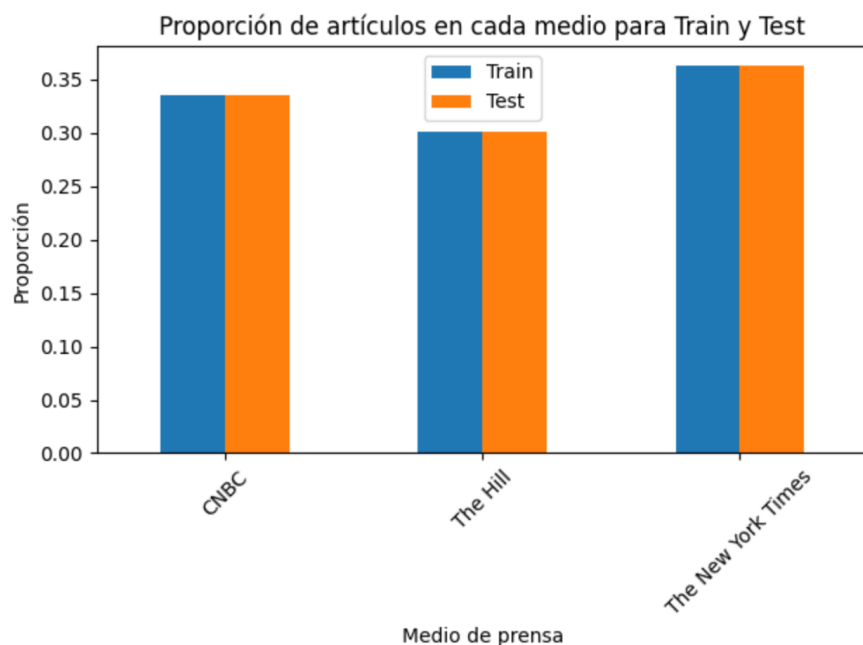
5. Cambio de medio de prensa

En esta parte se modificó el conjunto de datos reemplazando *Reuters* por *The Hill*, manteniendo *The New York Times* y *CNBC*. El objetivo fue analizar cómo cambia el desempeño del modelo cuando se trabaja con un conjunto de datos más balanceado entre las distintas clases.

En el problema original, *Reuters* representaba aproximadamente el 63% de los artículos, mientras que los otros dos medios tenían una cantidad considerablemente menor. En cambio, al utilizar *The New York Times*, *CNBC* y *The Hill*, las tres clases presentan un número de artículos mucho más similar, permitiendo evaluar el efecto que tiene el desbalance sobre la clasificación.

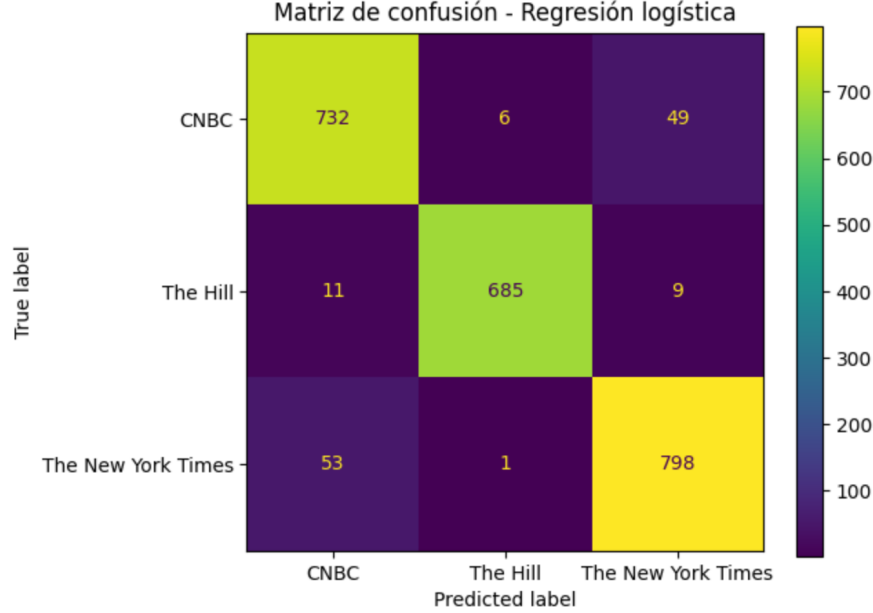
Para realizar la comparación, se repitió el mismo procedimiento utilizado anteriormente: se aplicó la función `clean_text()`, se realizó la partición estratificada en entrenamiento y prueba, se representaron los textos mediante TF-IDF y se entrenó el modelo de regresión logística, utilizando la misma configuración que obtuvo el mejor desempeño en la sección anterior.

Publication	Cantidad	Proporción (%)
The New York Times	2840	36.4
CNBC	2623	33.6
The Hill	2349	30.1



	Precision	Recall	F1-score	Support
CNBC	0.92	0.93	0.92	787
The Hill	0.99	0.97	0.98	705
The New York Times	0.93	0.94	0.93	852
Accuracy			0.94	2344
Macro avg	0.95	0.95	0.95	2344
Weighted avg	0.95	0.94	0.95	2344

Table 3: Accuracy: 0.94



Al reemplazar *Reuters* por *The Hill*, el conjunto de datos quedó considerablemente más balanceado entre los tres medios. En este escenario, el modelo de regresión logística alcanzó un accuracy de aproximadamente 0,94, superior al obtenido en el problema original.

A diferencia del caso anterior, las métricas de precision y recall son altas y bastante similares para las tres clases. Esto indica que el modelo no depende de predecir mayoritariamente una sola categoría, sino que logra identificar correctamente artículos de los tres medios.

La matriz de confusión muestra que *The Hill* es el medio mejor diferenciado, con muy pocos errores de clasificación. Las principales confusiones se dan entre *CNBC* y *The New York Times*, lo que puede deberse a cierta similitud temática o léxica entre algunos de sus artículos.

Estos resultados sugieren que el desbalance del problema original, generado principalmente por la gran cantidad de artículos de *Reuters*, influía en el desempeño del modelo y en la distribución de los errores. En problemas de clasificación con clases desbalanceadas, técnicas como el sobremuestreo de las clases minoritarias o el submuestreo de la clase mayoritaria podrían ayudar a reducir este efecto, aunque en este caso no fue necesario implementarlas.

6. Técnica alternativa de extracción de features

Una alternativa para representar texto son los word embeddings, como Word2Vec, GloVe o FastText. A diferencia de Bag of Words y TF-IDF, donde cada palabra se representa de forma independiente, los embeddings representan cada palabra mediante un vector numérico aprendido a partir de grandes corpus de texto.

La idea principal es que palabras que aparecen en contextos similares obtengan representaciones similares. De esta forma, el modelo puede capturar relaciones semánticas entre palabras, por ejemplo, identificando que car y automobile tienen significados parecidos, aunque sean palabras diferentes.

En comparación con Bag of Words o TF-IDF, se esperaría obtener una representación que capture mejor el significado y las relaciones entre las palabras, ya que incorpora información semántica del texto. Esto podría mejorar el desempeño de los modelos de clasificación, especialmente cuando distintos artículos expresan ideas similares utilizando vocabulario diferente. Sin embargo, estas técnicas suelen requerir un mayor costo computacional y, en muchos casos, modelos previamente entrenados sobre grandes cantidades de texto.

Notas

Durante la elaboración de este trabajo se utilizó inteligencia artificial como herramienta de apoyo en distintos aspectos técnicos del proyecto, principalmente para el desarrollo y corrección de código, comprensión de errores, manejo de conflictos entre dependencias y mejora de visualizaciones.

Repositorio de GitHub

<https://github.com/valentinatorres29/IntroCD.git>

Fuentes

IBM. What are Word Embeddings? IBM Think. <https://www.ibm.com/think/topics/word-embeddings>

scikit-learn. Cross-validation: evaluating estimator performance. https://scikit-learn.org/stable/modules/cross_validation.html

scikit-learn. GridSearchCV. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

scikit-learn. LogisticRegression. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

scikit-learn. MultinomialNB. https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html

scikit-learn. PCA. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>

scikit-learn. TruncatedSVD. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html>

scikit-learn. Working with text data. https://scikit-learn.org/stable/tutorial/text_analytics/working_with_text_data.html

TensorFlow. Word embeddings. https://www.tensorflow.org/text/guide/word_embeddings

Repositorio en github